

EMERGENCE PATTERNS VIA SYNCHRONIZATION

Adam Banham¹ , Claudia Szabo¹ 

¹ School of Comp. Sci. and Info. Tech., Adelaide University, Adelaide, South Australia, Australia

ABSTRACT

Understanding how emergent behavior appears and unfolds in complex adaptive systems (CAS) can aid their design, development, and deployment. Existing work focuses on the intricate modeling of micro- and macro-states that lead to emergent behavior, without considering the engineering of emergent behavior in a principled and reusable manner. In this paper, we focus on the potential of defining patterns of emergent behavior as collections and sequences of system states and CAS properties that facilitate the appearance of specific behaviors. We build on the practice of workflow patterns, which are extensively used in process mining and business process management to abstract specific behaviors and to facilitate reasoning about behavior under complex, sophisticated conditions. We propose a representation of emergent behavior patterns using Business Process Model and Notation (BPMN) and investigate the feasibility of this reasoning tool when analyzing several well-known models.

1 INTRODUCTION

Emergent behavior, which is malign or benign behavior that arises from non-linear interactions of heterogeneous agents, with effects across time and space, is inherent in Complex Adaptive Systems (CAS) and Systems of Systems (SoS). It manifests as system, macro- level properties that are not readily predictable from local rules alone and has been studied extensively across a variety of systems. While many studies show that emergent behavior can be observed in complex systems (Couzin and Krause 2003; Ballerini et al. 2008; Beer 2015; Szabo 2024), work on understanding how desired behaviors arise, how they unfold over time, and how they can be engineered, shaped, constrained, or stabilized through design is still ongoing (Mittal et al. 2018; Foguelman et al. 2021; Wilsdorf et al. 2023; Pecker-Marcosig et al. 2025).

Many studies focus on the bottom-up modeling of emergent behavior in complex systems (Chen et al. 2007; Chen et al. 2010; Mittal et al. 2018; Szabo et al. 2019; Szabo 2024), either through the use of supporting architectures (Banham et al. 2026) or formalisms such as DEVS (Mittal 2013; Zeigler et al. 2018; Foguelman et al. 2021; Pecker-Marcosig et al. 2025). Other works focus on creating repositories of emergent behavior with an understanding that these behaviors can be reused, either for identification of new emergent behaviors (Santos and Zhao 2017; Birdsey and Szabo 2014) or for their reuse (Wilsdorf et al. 2023; Silva et al. 2025). Lastly, several works focus on designing or studying systems that exhibit several properties that are 'co-located' with emergent behavior or similarly desired in a complex system, such as adaptation (Mittal 2014; Szabo et al. 2020), self-organisation (Birdsey and Szabo 2014; Serugendo et al. 2006; Prokopenko, Mikhail and Gershenson, Carlos 2014), and resilience (Szabo, Castro, Denil, and Sanchez 2023; Honhaga and Szabo 2024; Szabo, Castro, Semeshenko, and Mittal 2026). In addition, existing works highlight that specific conditions (Chen et al. 2010) or behavioral constructs (Haynes et al. 2017; Pecker-Marcosig et al. 2025) can potentially lead to emergent behavior, but the conceptual model of how these behavioral constructs appear or evolve is not understood.

In this paper, we focus on several CAS behaviors and properties that are co-located with emergent behavior. Our analysis of several well-known models that exhibit emergent behavior such as such as Conway's game of life (Gardner 1970; Rendell 2011), flock of birds (Reynolds 1987; Ballerini et al. 2008), and predator-prey (Chen et al. 2007; Raczyński 2020) has shown that each exhibits at least one instance of agents seemingly timing and sequencing their interactions without encoded simulation code, referred to as

synchronization (Pikovsky et al. 2001; Mahmoodi et al. 2024). Synchronization is an important element to consider as it requires interactions to occur at runtime at specific times and, together with other CAS properties such as self-organization (Couzin and Krause 2003), leads to emergent behaviour (Chen et al. 2007; Chen et al. 2010; Szabo 2024; Szabo et al. 2019). To the best of our knowledge, synchronization is not a property that is explicitly modeled in simulation studies. In this paper, we argue that having identifiable patterns for synchronization can open new pathways for the modeling, engineering, and understanding of complex adaptive simulations.

While modelers can attempt to emulate conditions that lead to synchronization, self-organization, and then emergent behavior, the process can be lengthy and often yields ad hoc approaches with suboptimal outcomes (Hadzikadic and Whitmeyer 2019; Uhrmacher et al. 2024). To empower modelers, we explore patterns (Sapin 2010; Russell et al. 2016) to provide a frame of reference and conceptual blueprints for emergent behavior, enabling modelers to identify when behaviors will lead to emergent behavior during the simulation run. In this paper, we present a first step at identifying a series of patterns as they appear in conceptual models derived from seminal work and consider these patterns and their associated models as reference models of emergent behavior.

The benefits of a series reference models are evident in the adoption of workflow patterns (van der Aalst et al. 2003; Russell et al. 2016). The use of workflow patterns in the disciplines of process mining (van der Aalst 2016) and business process management (Dumas et al. 2018) has enabled a generalized understanding of execution semantics for workflow processes, with synchronization as a key element. The acceptance of workflow patterns enabled industry success stories (Freund and Rücker 2019) like Celonis, Camunda, and SAP Signavio as well as facilitating academic discussion of highly complex and concurrent behaviour (Wynn 2006; van der Aalst 2016; Carmona et al. 2018; vom Brocke et al. 2024). By synthesizing patterns for emergent synchronization, we aim to empower modelers and researchers studying emergent behaviour (Uhrmacher et al. 2024; Mittal et al. 2018) in a similar vein. The contribution of our work is threefold. Firstly, we present several emergent synchronization patterns that facilitate a straightforward modeling of emergent behavior, with the aim of reducing study development time. Secondly, we use the BPM notation (Aggarwal et al. 2014) to describe constructs that may lead to synchronization (Russell et al. 2016) to facilitate the modeling of highly complex behavior. Thirdly, we present a case study analysis of the application of emergent synchronization patterns to a complex simulation model.

2 EMERGENT BEHAVIOR THROUGH SYNCHRONIZATION

This section introduces our definition of emergence and synchronization in agent-based simulations. Emergence is often described as agents organizing their interactions collectively, leading to the creation of an unpredictable macro, or system level, state that cannot be decomposed into further micro, or component level, states (Chen et al. 2010; Mittal et al. 2018; Szabo et al. 2019; Mahmoodi et al. 2024; Szabo 2024). In this work, we consider emergence to be a consequence of at least one synchronization, resulting in previously unencoded behavior. We now introduce our definition of *emergence*.

Definition 1 (Emergence) An emergent behavior, or an emergence, is a collection of agents and their interactions that leads to an observable simulation macro-state that cannot be reduced to further subcomponents or micro-states.

For the macro-state to emerge, there needs to be a mechanism within the simulation to organize or coordinate agent interactions, referred to as synchronization. The mechanisms that govern these synchronizations influence agents' future interactions in many ways, such as through state sharing or proximity to other agents. A possible consequence of a synchronization mechanism is that agents self-organize (Reynolds 1987; Couzin and Krause 2003; Ballerini et al. 2008; Raczyński 2020) and seemingly coordinate and adapt to environmental changes without any direct encoding in the simulation code. Moreover, synchronization is the mechanism by which agents can coordinate and describes the conditions needed for the coordination.

Definition 2 (Synchronization) Synchronization can occur when two or more agents share access to a simulation state (read or write) or require their participation in an interaction. A synchronization exists for some period of a simulation and, when stabilized, can lead to emergent phenomena. Lastly, a synchronization is sustained by agent interactions and will decay in their absence.

In this paper, we present patterns to identify points of synchronization that may lead to emergent behavior. These model synchronization points are intended to guide designers and engineers and may not indicate correlation. Furthermore, we argue that without opportunities for synchronization, emergent behavior must be directly encoded.

2.1 Modeling Interactions and Synchronizations

In this paper, we present our patterns and models using the Business Process Model and Notation (BPMN) 2.0 (Aggarwal et al. 2014). BPMN is typically used to model the internal processes of an organization, where models describe the behavior that occurs in response to an initial event, leading to a desirable state or to a new event. We chose BPMN for three main reasons. Firstly, BPMN was designed for the catching and throwing of events, creating a clear flow of behavior in response, the data requirements of behavior, and has been used extensively to describe and analyze concurrent behaviour (van der Aalst et al. 2003; Wynn 2006; Russell et al. 2016). The main constructs that facilitate this are gateways, sub-processes, tasks, events, and associations. Secondly, BPMN can describe interactions between organizations or separate subsystems and provides declarative process flows via ad hoc, sequential, and parallel task types to model agent interactions in simulations. Thirdly, by synthesizing our proposed patterns in BPMN, the execution semantics of our patterns can be understood at design time, regardless of the simulation environment, using BPMN’s formal execution semantics (Aggarwal et al. 2014). Together, these reasons mean that BPMN provides a notation to describe agent classes, their interactions, and to encapsulate the decision logic for interactions in a single model.

A further benefit of using BPMN is that model verification and validation have been extensively studied within the discipline of process mining (van der Aalst 2016; Carmona et al. 2018; vom Brocke et al. 2024). One key component of such analysis is that the semantics of BPMN models (Aggarwal et al. 2014; Dumas et al. 2018) can be readily translated into lower-level formalisms, such as coloured Petri nets (Dechsupa et al. 2018) or classic Petri nets (Dijkman et al. 2008). In addition, this transformation means the semantics can be converted into executable code for discrete-event simulations, such as Petri net-based (Dijkman 2024) simulations, or into other formalisms, such as DEVS (Chow and Zeigler 1994; Zeigler et al. 2018). Thus, our decision to model in BPMN provides a visual, transformable modeling formalism for describing agent interactions and enables verification, by avoiding the state-explosion problem (van der Aalst 2016) that often occurs in lower-level formalisms.

2.2 BPMN 2.0 Constructs

In this paper, we use elements of the BPMN 2.0 notation to describe agent interactions and provide a primer for the unfamiliar reader. **Tasks.** The specification (Aggarwal et al. 2014) describes a task as an atomic activity within a process, which generally is not broken down to a finer level of detail. These are generally drawn as rectangles, with the task name in the middle. When work can be broken down further, we can use a sub-process task ($\boxplus$) to encapsulate these steps into a single task, and, when expanded, the inner elements are shown (as in Figure 1). Two additional modifiers can be applied to the sub-process or task to further modify the execution semantics. Firstly, the sub-process can be “ad-hoc” ($\sim$), meaning that the element within can be executed in any order. Secondly, we can say that the task is executed in a parallel, multi-instance manner ($|||$). In our agent-based modeling setting, a task/sub-process is an interaction or action undertaken by an agent. For instance, a wolf can execute the “move” or “die” action. Then, we might describe instances of wolves alive at a point in the simulation that can execute a range of actions through an expanded ad hoc sub-process, executed in a multi-instance manner ($|||\sim$) as in Figure 6. **Data Objects.** The

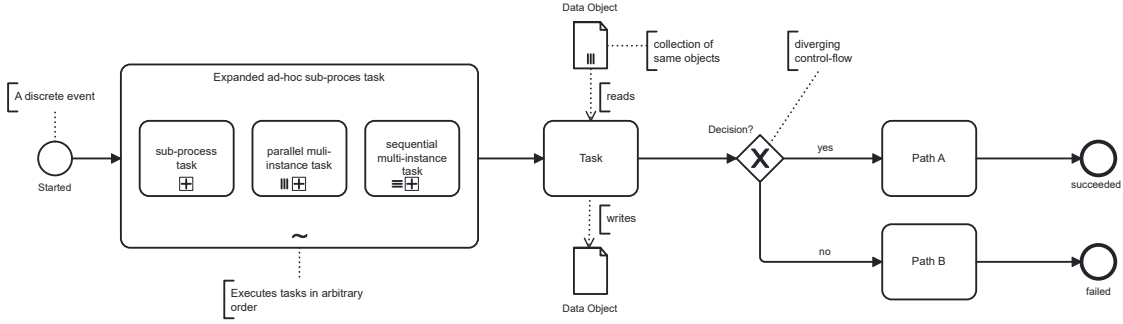


Figure 1: Highlighting the various notations for tasks in BPMN 2.0 used for our patterns.

specification (Aggarwal et al. 2014) describes data objects as providing information for performing tasks or what information they produce/changed after executing. These elements represent the state of something in our agent-based modeling setting, e.g., a wolf’s location or age in a simulation. These are typically drawn as a dog-eared page, as seen in Figure 1, and include a meaningful label for the information below the data object. **Associations.** The specification (Aggarwal et al. 2014) describes associations as a way to link information with modeling elements, i.e., to say that a task requires some data, or that it changes/produces information. These are dashed directed arcs, where the direction describes whether the association uses or produces. If the direction points to a data object, the source of the association produces that information, as in Figure 1. Solid directed arcs denote that elements have connected execution semantics, as in Figure 1. **Events.** The specification (Aggarwal et al. 2014) describes events as a description of a moment that happens during the course of a process. Events are intended to be generic and all-encompassing; in the context of simulation studies, they can describe discrete moments or messages. These are circles with labels describing what happened, as shown in Figure 1. **Gateways.** The specification (Aggarwal et al. 2014) describes gateways as elements that control the flow of the process and when it converges or diverges during execution. These gateways are shown as diamond nodes. There are many types of gateways, each with its own semantics for how it converges or diverges flows. For instance, an exclusive gateway considers conditions associated with outgoing flows and then executes one of the outgoing flows based on valid conditions, as shown in Figure 1.

3 PATTERNS OF EMERGENT SYNCHRONIZATION

Drawing on the motivation from workflow patterns (van der Aalst et al. 2003; Russell et al. 2016), which aimed to identify useful routing constructs and establish how they are used in the state-of-the-art, we derive and propose patterns of emergent synchronization for simulation modelers. We approached the derivation of our patterns constructively, building on Def. 1 and Def. 2 and using seminal emergent behavior examples as guidelines.

Each derived pattern highlights a pattern for modelers to design and apply emergence within their studies. A key component of our work is deriving the semantics of emergence, enabling any simulation study to leverage our proposed patterns. The following patterns describe these semantics by considering how synchronizations in seminal models (Gardner 1970; Reynolds 1987; Raczynski 2020) can lead to emergence. This work presents a first step towards this derivation, and we note here that these patterns are not all-encompassing and are intended to encourage discussion of emergence in simulation studies.

In the following, we focus the definition of synchronization patterns on several elements, including agent states, the type of interaction between agents (e.g., one agent reading another agent’s state), and temporal aspects of interaction. We refer to a set of agents as the set A , instances of agents as members $a \in A$, a grouping of agents as subsets $g \subseteq A$, and populations of agents as exclusive subsets $P_1 \cup P_2 = A \wedge P_1 \cap P_2 = \emptyset$.

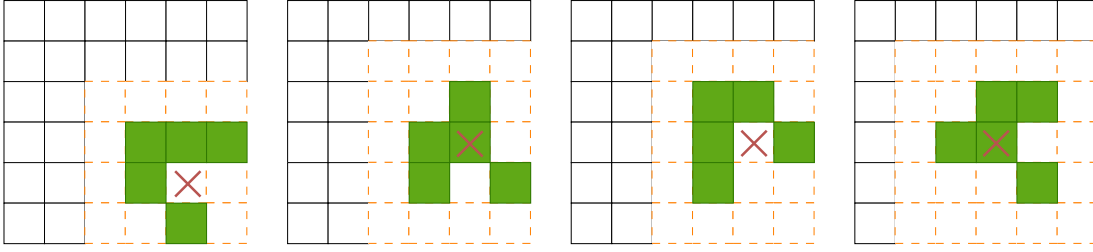


Figure 2: The visual representation of the glider pattern. The red cross shows where the source entity is across the pattern. The full set of dependent entities for the glider is shown in orange-dashed cells.

3.1 Periodic Emergence

The first pattern we derive is *periodic emergence*, which focuses on situations where agents' states repeat themselves periodically during a simulation run. In many simulation studies, it may be of interest to model seasonal influences, such as highs and lows in environmental resources, or to allow for the continuous cycle through states, i.e., steady into busy into burnt-out, then back to steady. A periodic emergent synchronization (or periodic emergence for short) allows for a large class of influences without the need to encode all possibilities. Furthermore, it can increase the diversity of states observed across replications, as the synchronizations that lead to such emergence often arise from the simulation's initial conditions. An example of periodic emergence can be found in Conway's Game of Life (Gardner 1970).

Conway's Game of Life (Gardner 1970) (GoL) is a cellular automaton simulation, where each cell has an internal state indicating whether it is alive. The simulation applies a set of rules to each cell, based on the current states of its surrounding cells, to determine the next state of the cell. A well-known emergent behavior is the formation of these cells into spaceships (Bays 2010; McIntosh 2010) that can travel across the world pane in the simulation, while other instances of synchronizations may lead to still lifes and oscillators (Bays 2010; McIntosh 2010; Rendell 2011).

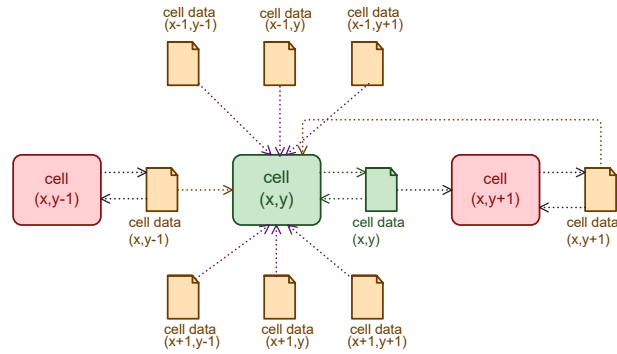


Figure 3: GoL interactions: using BPMN 2.0, tasks represent actions, data objects represent internal states, and arcs denote dependencies and outputs on these data objects.

We now consider what interactions may be contributing to possible synchronization in the simulation, as shown in Figure 3. The mechanism of synchronization is the sharing of data between cells for determining the outcome of interactions. We modeled this cell interaction through a (green) task `cell(x,y)` with *read* associations to the data objects of other cells and a *write* association to its own data object (green data object). These associations model that a cell's output depends on the internal states of surrounding cells, which, in turn, depend on their surrounding cells (yellow elements), leading to system-wide properties. If the synchronization stabilizes, then a group of cells can form a self-sustaining closed system, leading to emergent behavior as a repeating pattern in the cells' state. While patterns in lifeform classes in GoL models have been studied by Sapin (2010) and Beer (2015), these studies have often not considered emergence nor the synchronization conditions or patterns that led to it. Building upon GoL, we propose a pattern for introducing this periodic characteristic of interaction in Pattern 1.

Pattern 1 (Periodic Emergence). A synchronization may arise if a set of agents, A , have interdependencies among their internal states. The synchronization is bounded around a source agent $s \in A$, and all agents $a \in A$ read from some subset of agent states $\hat{A} \subseteq A$ to inform their next state. If the synchronization leads to a repetition in the agents' states, then a periodic emergence is observed. We define a periodic emergence as 'in-motion' if the source agent s changes over time. If the source agent does not change after emergence is observed, we define the periodic emergence as 'stationary'.

For instance, an in-motion periodic emergence in GoL could be the forming of spaceships or gliders, while a stationary emergence could be a still life or an oscillator. Expanding on an in-motion periodic emergence, we consider a 'glider', a spaceship consisting of all entities within a 5v5 grid surrounding the source entity. In Figure 2, we highlight the evolution of the glider through a repetition with a period of 4 cycles to visualize the periodic emergence. It is important to note that in the case of cell automata it is straightforward to see both the internal state and the repeating pattern, since the state is often binary. It is also possible to model the entire system without explicitly considering periodic emergence. However, in more complex systems, identifying the state being synchronized and the pattern being repeated may not be trivial, but the modeling as periodic emergence can potentially support the modeling exercise.

3.2 Topological Emergence

The second pattern we derive is *topological emergence*, which focuses on agents converging to their states based on their proximity to other agents. Many studies have considered how herds or flocks form in nature (Reynolds 1987; Olfati-Saber 2006; Cavagna et al. 2010), where the simulation needs to encode that at any point a group of agents will synchronize and emerge as an aggregated agent. A topological emergent synchronization (or topological emergence for short) appears as a topological arrangement of agents for a duration of the simulation, which can have benefits for members of the arrangement, such as protecting from predators (Ballerini et al. 2008; Cavagna et al. 2010). We derived topological emergence from Reynolds (1987) flocks of birds simulation.

When modeling and simulating flocks of birds (FoB), the challenge is often how to best achieve the synchronized group behavior to make the flock seem like a fluid whole. Reynolds (1987) describes that the fluid motion must come from the aggregate result of agents' actions, opposed to a centralized control mechanism, and makes several observations to inform their modeling of interactions for birds: birds have a desire to stay close to a flock; birds desire to avoid collisions; and birds do not pay attention to all flockmates in a flock. For a flock to emerge from the interactions among birds, some synchronization between agents must occur.

We now consider what interactions may be contributing to the synchronization leading to flocks in the simulation (Reynolds 1987; Couzin and Krause 2003; Katz et al. 2011), as shown in Figure 4. A boid is a simulated agent within the flock of birds simulation, introduced to add new modifiers to an aggregated flight trajectory, such as flying near flockmates, avoiding collisions, and matching velocity. We model the internal state of a boid as a (green) task `Bird(B1)` that sets its state, i.e., a (green) flight trajectory data object, in some random fashion. The synchronization mechanisms are modeled as yellow tasks (`centering`, `avoidance`, `velocity matching`) that have *read* associations with the closest trajectories and *write* associations with the (green) flight trajectory of the boid. Notably, these tasks would update the boid's

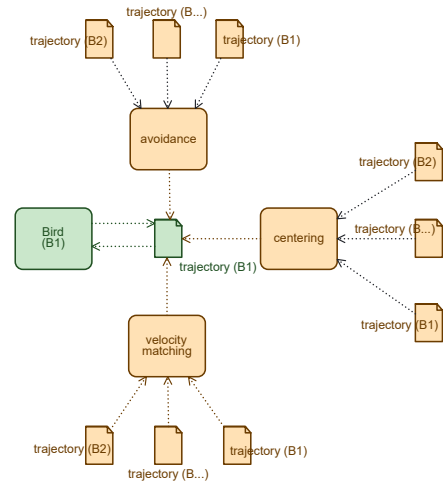


Figure 4: The micro (green) and macro interactions (yellow) derived from the flock of birds presented.

flight trajectory additively and may not act if there are not three other boids in close proximity, i.e., the associations describe hard constraints on the tasks being executed. If the synchronization stabilizes, then emergent behavior may occur and flocks may be observed in the simulation.

Unlike GoL, FoB entities are unlikely to exhibit a repeating pattern in their internal state. Instead, emergent behavior appears as a follow-up to a topological structure (Ballerini et al. 2008; Olfati-Saber 2006), in which the group reaches consensus on a direction, and members align their states with it. Notably, the consensus builds iteratively over time, allowing it to change when the group encounters new stimuli, e.g., new flock members or predator attacks (Ballerini et al. 2008; Cavagna et al. 2010). Building upon FoB, we proposed a pattern for introducing our topological emergence in Pattern 2.

Pattern 2 (Topological Emergence). *A synchronization may arise if a set of agents A has a subset of interactions that inform their state based on a topological grouping of these agents $g \subseteq A$. The synchronization is reinforced through the agents' interactions that remain in proximity to others, and when stabilized, can lead to topological emergence. Topological emergence describes a state where all members of the group align their states to stabilize the arrangement over the course of the simulation. We define a topological emergence as 'growing' if the number of agents in the topology increases over time. If the number of agents are decreasing over time, we define the topological emergence as 'decaying'.*

For instance, consider the boids and 'flocks' in Figure 5, where many synchronizations have stabilized, leading to a topological emergence that formed a consensus between boids. We can see that the structure of the topology is not fixed, but given the typical rules (Reynolds 1987; Ballerini et al. 2008; Olfati-Saber 2006) for adjusting the birds' trajectories, the topology is reinforced in a general direction. It is important to note that, in FoB, additive influences on a boid's state during synchronization enabled topological emergence, rather than interactions fully determining the state, as in GoL.

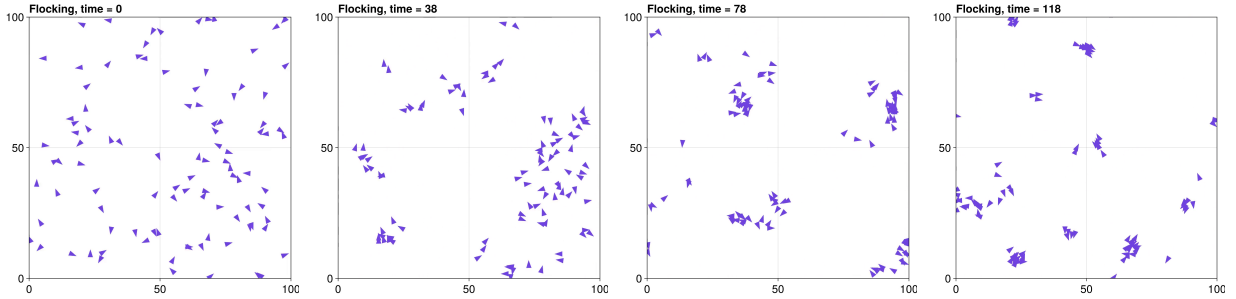


Figure 5: Topological emergent synchronization in a flock of birds using Agents.jl (Datseris et al. 2022).

3.3 Symbiotic Emergence

The third pattern we derive is *symbiotic emergence*, which focuses on restricting or enabling interactions for agents classes or groups. Many studies need to consider whether classes of agents stabilize during a simulation or collapse (Lotka 1910; Raczynski 2020; Cropp and Norbury 2015) as a result of interactions taken by one of the agent classes. These interactions often consider the proximity between agents and others of their class, enabling or restricting their behavior in some way. A symbiotic emergent synchronization (or symbiotic emergence for short) appears within the population of one or more classes of agents, driven by interactions that require participation from agents across different classes. An example of symbiotic emergence can be found in Predator-Prey models (Chen et al. 2007; Chen et al. 2010; Raczynski 2020).

Predator-Prey (PaP) models consider the interactions between two populations of agents: predator and prey. A classic example is the modeling of wolf and rabbit populations (Lotka 1910; Raczynski 2020; Cropp and Norbury 2015). The rate of change for the wolves w (predator) and rabbit r (prey) populations over time t can be described using the Lotka-Volterra equations (Raczynski 2020; Cropp and Norbury 2015): $\frac{\Delta w}{\Delta t} = crw - dw$, $\frac{\Delta r}{\Delta t} = ar - bw$. Where a represents the intrinsic growth rate of rabbits, b is the predation rate, c is the increase of wolves due to predation, and d is the wolf mortality rate. These equations

do have some limitations and caveats to observe, such as stability and cyclic patterns (Cropp and Norbury 2015). However, by adopting an agent-based approach to PaP models, simulations can incorporate features that are difficult to capture with differential equations (Raczynski 2020) such as agent proximity.

PaP studies using agent-based models (Chen et al. 2007; Chen et al. 2010; Raczynski 2020) have considered the following rules for agents: (i) if alive, both wolves and rabbits move in a random direction; (ii) if a wolf is near a rabbit, the wolf will prey on the rabbit; (iii) each time a wolf’s predation action is successful, a new wolf is born after some delay and (iv) wolves may die in each generation, and every rabbit may give birth in each generation. Using these rules, Chen et al. (2010) investigated whether their simulation would lead to population synchronization or to the predator dying out from starvation.

We generalize the approach taken by Chen et al. (2010), and say that each class of agent in PaP has its own class-specific interactions in Figure 6, and consider the mechanisms for synchronization. We modeled the populations of wolves and rabbits as parallel ad-hoc sub-process tasks, where wolves can move and die, and rabbits can move and eat. All of these interactions modify the agents’ states through a write association to the `instance` data object.

The key mechanism for the synchronization is the (yellow) `predation` task, which requires an instance of each population to be in proximity to each other. For the wolves to survive in the simulation, both their population and the rabbit population need to stabilize. In comparison to GoL and FoB models, not only are entities and their actions not homogeneous, but some actions require that instances from both classes of agents be “nearby” to each other to perform said action.

The difference between PaP and previous examples, i.e., GoL and FoB, is the focus on populations of agents that are linked through inter class-level interactions. The contextual requirements for the interaction to occur must emerge within the simulation, i.e. the proximity between a wolf and a rabbit. Through the interaction, i.e., `predation`, the synchronization may stabilize, leading to the observation that neither the wolf nor the rabbit population collapses for the duration of the synchronization. Additionally, the synchronization ensures that the contextual requirements are met frequently enough for the interaction to occur. Building upon PaP, we proposed a pattern to capture class-related synchronization in Pattern 3.

Pattern 3 (Symbiotic Emergence). *A synchronization may arise if a set of agents A can be split into separate populations $A = P_1 \cup P_2$, and an inter-class interaction i requires agents from different populations, i.e. $a_a \in P_1 \wedge a_b \in P_2$ to be enacted. The synchronization is reinforced by interactions among the populations, and, if stabilized, it will enable or disable the inter-class interaction i to observe symbiotic emergence. Symbiotic emergence describes specific trends in the populations of agent classes arising from the presence or absence of inter-class interaction i . We define a ‘stabilizing’ symbiotic emergence as one in which neither population collapses to zero for its duration. If a population collapses, then the symbiotic emergence is defined as ‘unstable’.*

For instance, consider the snapshots of an unfolding PaP model in Figure 7. An important consideration for PaP models is the choice of initial conditions, as many will not lead to stabilizing symbiotic emergence. A similar observation holds for GoL models, where the initial conditions determine whether the cell population collapses. Predation prevents rabbits from depleting their food source and collapsing, while the wolf population focuses on a small herd of rabbits due to the constant availability of rabbits.

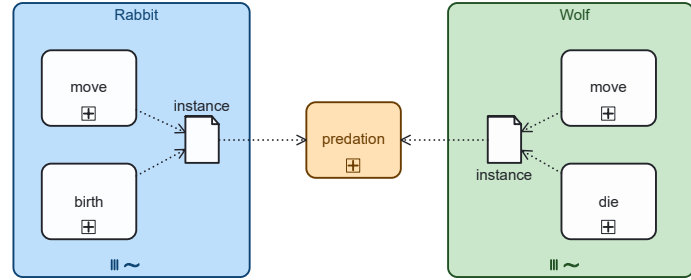


Figure 6: The class-specific (green and blue) and inter-class interactions (yellow) derived from the predator-prey models by Chen et al. (2010), Raczynski (2020).

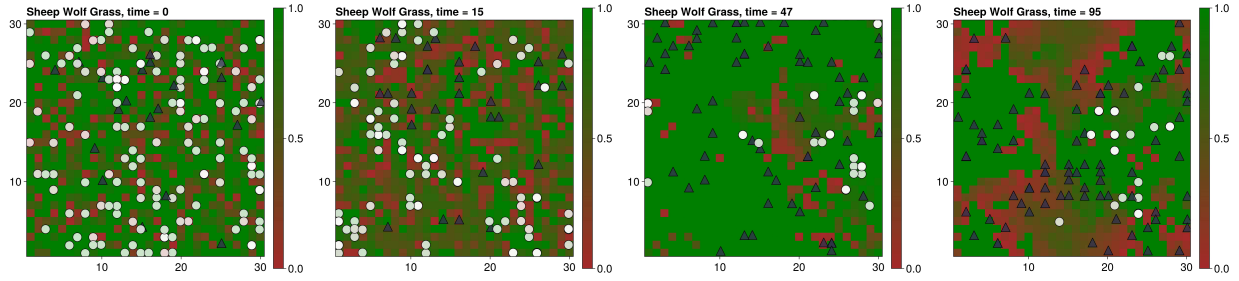


Figure 7: Symbiotic emergent synchronization in a predator-and-prey model using Agents.jl (Datseris et al. 2022). Food for sheep is shown as a color gradient from green (ample) to red (lacking).

4 CASE STUDY ANALYSIS

In this section, we apply our proposed patterns to discuss the modeled behaviors in a recent simulation study (Banham et al. 2026). The study simulates a board game of ‘Risk’, exploring several player strategies: random, defensive, and aggressive. A player’s turn progresses as follows: first, they place troops on territories, then they can attack unowned territories, and finally, they can transfer troops between connected owned territories. The goal of the game is to own all the territories. We represent the different strategies as distinct agent classes and highlight their interactions in Figure 8. Our discussion focuses on the attack action between players and whether they lead to emergent simulation states.

In the following examples, the internal states of each agent are their territories and the number of troops in each territory. We first consider the attack strategy for the random class of players. The random player strategy simply chooses its attacks on other players at random, and the outcomes of these attacks affect both the attacker and defender. This random decision-making means the interactions cannot build up any synchronization. Without reinforcement of synchronizations, emergences are brief, if at all; aligning with our patterns as they encourage the introduction of informed decision logic for stabilizing synchronizations.

4.1 Defensive

The defensive strategy focuses on making safer attacks; it attacks only if an owned territory has three times as many troops as the defender. For our discussion, the `attack` interaction is an inter-class interaction that requires proximity to other players to occur. In replications, a synchronization was driven by the `place` and `transfer` interactions, either enabling or disabling `attack`. Especially since the strategy spreads its troops evenly across its frontline territories, making it harder to enable `attack`, furthering the synchronization to stabilization. As `attack` has a destabilizing effect on agents, the lack of interaction means that the population of troops on each side may never collapse. Emergent from this synchronization was a stabilizing symbiotic emergence when a defensive strategy encloses an opposing territory (Figure 9a). This emergence often led to deadlocks in replications, where no player could be deemed the winner. Using our theory and patterns, in order to decay the synchronization, we would need to introduce a random chance of an `attack` occurring or provide an alternative pathway to trigger `attack`.

4.2 Aggressive

The aggressive strategy attempts to create breakaways into unowned territories through a chain of attacks. To enable this strategy, the agent will move all connected troops to a single territory to prepare as shown in Figure 9b. From the read and write associations in the model, we can identify that the `attack` has interdependencies and proximity conditions with other players. These interdependencies involve reading the other territories to move their troops and position them for the next chain of attacks, thereby feeding synchronization. The synchronization can lead to topological emergence, creating a repeating ouroboros pattern in which both are chasing each other, as shown in Figure 9b. Using the proposed pattern, we can identify at design time an opportunity for two agents to coordinate through shared information and

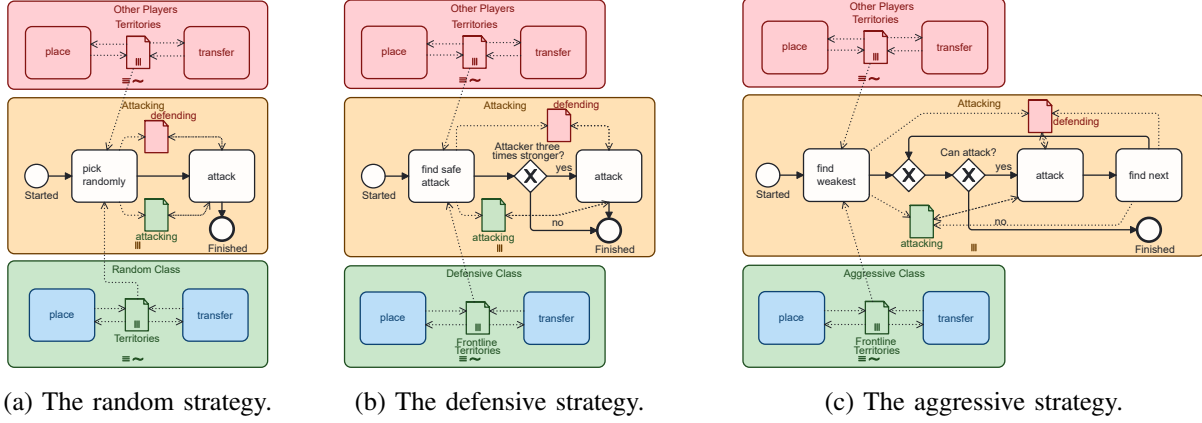
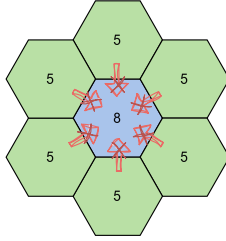
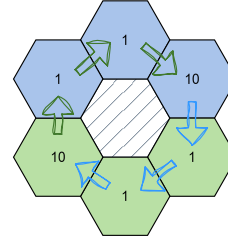


Figure 8: Interactions between three risk strategies. Data objects represent territories owned by a player.



(a) The simulation state for the symbiotic emergence, leading to a deadlock with no winner.



(b) The simulation state for the topological emergence, where agents chase each other in a spiral.

Figure 9: Different emergent states arising from interactions between agent classes.

interactions, leading to emergent behavior. Notably, in this case, an aggressive agent will always do at least one `attack` or position to find that one attack, so we don't see the synchronization in the defensive case.

We envisage the use of these synchronization patterns in two ways. Firstly, formalizations of these patterns could be used as building blocks when modelers are developing models, reducing the need to model and prescribe interactions in detail, as the use of the patterns will require fewer conceptual symbols to describe desirable behavior. Secondly, the use of BPMN has the advantage that it allows for the identification of potential synchronizations and dependencies among interactions through the use of dedicated tools such as Cardanit (for verification) or VBPMN (for validation). For instance, the associations across task boundaries in all strategies indicate that the first step within `attack` requires access to other agents' internal states to select territories to attack. The dependencies also highlight that the selection is taken from a collection for its decision logic via data object collections. So if the collection on either side is empty, the decision logic deadlocks, and no further interactions occur. To highlight branching in the logic, gateways can be used to model conditional interactions as shown in Figures 8b & 8c.

A challenge for future work is determining how these patterns may help with the engineering of emergent behavior. In addition, it is important to note that these patterns do not prescribe what emergent behavior might be, nor its impact on the system. Their value, following our inspiration from workflow patterns (Russell et al. 2016), lies in clarifying useful routing constructs that enable emergence in simulation studies without mandating implementation. Future work could expand on the intent and descriptions of our patterns to test their combination and reapplication in large-scale complex systems simulations.

5 CONCLUSION

This paper introduced the notion of synchronizations as a precursor to emergent behavior, as agent interactions occur and unfold. To help modelers understand and engineer emergent behavior, we presented three patterns

that focus on abstractions of time, space, and class-based constraints on agent interactions, and applied them to a recent study. We envisage that these patterns will help designers create models for simulation studies with synchronization in mind.

REFERENCES

- Aggarwal, A., M. Amend, S. Astier, A. Barros, R. Bartel, M. Benitez, *et al.* 2014. “BPMN™ — Business Process Model and Notation”. Standard, Object Management Group (OMG).
- Ballerini, M., N. Cabibbo, R. Candelier, A. Cavagna, E. Cisbani, I. Giardina, *et al.* 2008. “Interaction ruling animal collective behavior depends on topological rather than metric distance: Evidence from a field study”. *Proceedings of the National Academy of Sciences of the United States of America* 105(4):1232 – 1237.
- Banham, A., R. Beruldsen, and C. Szabo. 2026. “On the Modelling of Aggregated Behaviour for Simulation: An Event-Based Architecture”. In *SIGSIM Principles of Advanced Discrete Simulation*. June 24–26, Vienna, Austria, 2026: ACM.
- Bays, C. 2010. *Introduction to Cellular Automata and Conway’s Game of Life*, 1–7. London: Springer London.
- Beer, R. D. 2015. “Characterizing Autopoiesis in the Game of Life”. *Artif. Life* 21(1):1–19.
- Birdsey, L., and C. Szabo. 2014. “An architecture for identifying emergent behavior in multi-agent systems”. In *Proceedings of the 2014 international conference on Autonomous agents and multi-agent systems*, 1455–1456.
- Carmona, J., B. F. van Dongen, A. Solti, and M. Weidlich. 2018. *Conformance Checking - Relating Processes and Models*. Springer.
- Cavagna, A., A. Cimorelli, I. Giardina, G. Parisi, R. Santagati, F. Stefanini *et al.* 2010. “Scale-free correlations in starling flocks”. *Proceedings of the National Academy of Sciences of the United States of America* 107(26):11865 – 11870.
- Chen, C., S. B. Nagl, and C. D. Clack. 2007. “Specifying, detecting and analysing emergent behaviours in multi-level agent-based simulations”. In *Proceedings of the 2007 Summer Computer Simulation Conference*, 969–976. California, USA.
- Chen, C.-C., C. D. Clack, and S. Nagl. 2010. “Identifying Multi-Level Emergent Behaviors in Agent-Directed Simulations using Complex Event Type Specifications”. *SIMULATION* 86(1):41 – 51.
- Chow, A. C. H., and B. P. Zeigler. 1994. “Parallel DEVS: a parallel, hierarchical, modular, modeling formalism”. In *Proceedings of the 26th conference on Winter simulation, WSC 1994*, 716–722. December 11–14, Lake Buena Vista, FL, USA, 1994.
- Couzin, I. D., and J. Krause. 2003. “Self-Organization and Collective Behavior in Vertebrates”. *Advances in the Study of Behavior* 32:1 – 75.
- Cropp, R., and J. Norbury. 2015. “Population interactions in ecology: A rule-based approach to modeling ecosystems in a mass-conserving framework”. *SIAM Review* 57(3):437 – 465.
- Datseris, G., A. R. Vahdati, and T. C. DuBois. 2022, jan. “Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity”. *SIMULATION* 100(10):1019–1031.
- Dechsupa, C., W. Vatanawood, and A. Thongtak. 2018. “Transformation of the BPMN Design Model into a Colored Petri Net Using the Partitioning Approach”. *IEEE Access* 6:38421–38436.
- Dijkman, R. M. 2024. “SimPN: A Python Library for Modeling and Simulating Timed, Colored Petri Nets”. In *22nd International Conference on Business Process Management*, CEUR Workshop Proceedings, 71–75.
- Dijkman, R. M., M. Dumas, and C. Ouyang. 2008. “Semantics and analysis of business process models in BPMN”. *Inf. Softw. Technol.* 50(12):1281–1294.
- Dumas, M., M. L. Rosa, J. Mendling, and H. A. Reijers. 2018. *Fundamentals of Business Process Management*. Springer.
- Foguelman, D., P. Henning, A. Uhrmacher, and R. Castro. 2021. “EB-DEVS: A formal framework for modeling and simulation of emergent behavior in dynamic complex systems”. *Journal of Computational Science* 53.
- Freund, J., and B. Rücker. 2019. *Real-Life BPMN*. 4 ed. Camunda.
- Gardner, M. 1970. *The Fantastic Combinations of John Conway’s New Solitaire Games*. Mathematical Games.
- Hadzikadic, M., and J. Whitmeyer. 2019. *Lessons from a project on agent-based modeling*. John Wiley & Sons, Ltd.
- Haynes, C., M. Luck, P. McBurney, S. Mahmoud, T. Vitek, and S. Miles. 2017. “Engineering the emergence of norms: A review”. *Knowledge Engineering Review* 32:1 – 31.
- Honhaga, I., and C. Szabo. 2024. “A simulation and experimentation architecture for resilient cooperative multiagent reinforcement learning models operating in contested and dynamic environments”. *Simulation* 100(6):563–579.
- Katz, Y., K. Tunström, C. C. Ioannou, C. Huepe, and I. D. Couzin. 2011. “Inferring the structure and dynamics of interactions in schooling fish”. *Proceedings of the National Academy of Sciences* 108(46):18720 – 18725.
- Lotka, A. J. 1910. “Contribution to the theory of periodic reactions”. *Journal of Physical Chemistry* 14(3):271 – 274.
- Mahmoodi, K., S. E. Kerick, P. J. Franaszczuk, T. D. Parsons, P. Grigolini, and B. J. West. 2024. “Complexity synchronization in emergent intelligence”. *Scientific Reports* 14(1):6758.
- McIntosh, H. V. 2010. *Conway’s Life*, 17–33. London: Springer London.
- Mittal, S. 2013. “Emergence in stigmergic and complex adaptive systems: A formal discrete event systems perspective”. *Cognitive Systems Research* 21:22 – 39.

- Mittal, S. 2014. "Model engineering for cyber complex adaptive systems". In *European Modeling and Simulation Symposium*.
- Mittal, S., S. Diallo, and A. Tolks. 2018. *Emergent Behavior in Complex Systems Engineering: A Modeling and Simulation Approach*. Wiley.
- Olfati-Saber, R. 2006. "Flocking for multi-agent dynamic systems: Algorithms and theory". *IEEE Transactions on Automatic Control* 51(3):401 – 420.
- Pecker-Marcosig, E., J. F. Presenza, I. Mas, J. I. Alvarez-Hamelin, J. I. Giribet, and R. Castro. 2025. "Simulation of Decentralized Coordination Strategies for Networked Multi-Robot Systems with Emergent Behavior-DEVS". In *Winter Simulation Conference*, 2740–2751. IEEE.
- Pikovsky, A., M. Rosenblum, and J. Kurths. 2001. *Synchronization: A Universal Concept in Nonlinear Sciences*. Cambridge Nonlinear Science Series. Cambridge University Press.
- Prokopenko, Mikhail and Gershenson, Carlos 2014. "Entropy methods in guided self-organisation".
- Raczynski, S. 2020. *Prey-Predator Models Revisited: Uncertainty, Herd Instinct, Fear, Limited Food, Epidemics, Evolution, and Competition*, 107–132. Singapore: Springer Singapore.
- Rendell, P. W. 2011. "A Universal Turing Machine in Conway's Game of Life". In *2011 International Conference on High Performance Computing & Simulation, HPCS 2012*, edited by W. W. Smari and J. P. McIntire, 764–772. Istanbul, Turkey.
- Reynolds, C. W. 1987. "Flocks, herds and schools: A distributed behavioral model". In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH*, 25–34. California, USA.
- Russell, N., W. M. P. van der Aalst, and A. H. M. ter Hofstede. 2016. *Workflow Patterns: The Definitive Guide*. MIT Press.
- Santos, E., and Y. Zhao. 2017. "Automatic emergence detection in complex systems". *Complexity* 2017.
- Sapin, E. 2010. *Gliders and Glider Guns Discovery in Cellular Automata*, 135–165. London: Springer London.
- Serugendo, G. D. M., M.-P. Irit, and A. Karageorgos. 2006. "Self-organisation and emergence in MAS: An overview". *Informatica* 30(1).
- Silva, K. C., A. R. G. Filho, A. Kumari, F. Horita, and V. V. G. Neto. 2025. "Towards Repositories of Emergent Behaviors: an Initial Repository and a Supporting Tool". *International Conference on System of Systems Engineering, SoSE* (2025).
- Szabo, C. 2024. "Complex Systems Modeling and Analysis". In *Winter Simulation Conference, WSC 2024*, 1–14. December 15–18, Orlando, FL, USA, 2024.
- Szabo, C., R. Castro, J. Denil, and S. M. Sanchez. 2023. "Resilience and Complexity in Socio-Cyber-Physical Systems". In *2023 Winter Simulation Conference (WSC)*, 660–670. IEEE.
- Szabo, C., R. Castro, V. Semeshenko, and S. Mittal. 2026. *Resilient Complex Adaptive Systems*. Wiley.
- Szabo, C., J. Groot, T. McAtee, and C. Pyromallis. 2019. "Who Did This: Identifying Causes of Emergent Behavior in Complex Systems". In *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, 262–270. Xiamen, China.
- Szabo, C., B. Sims, T. McAtee, R. Lodge, and R. Hunjet. 2020. "Self-adaptive software systems in contested and resource-constrained environments: Overview and challenges". *IEEE Access* 9:10711–10728.
- Uhrmacher, A. M., P. I. Frazier, R. Hähnle, F. Klügl, F. Lorig, B. Ludäscher, et al. 2024. "Context, Composition, Automation, and Communication: The C²AC Roadmap for Modeling and Simulation". *ACM Trans. Model. Comput. Simul.* 34(4):23:1–23:51.
- van der Aalst, W. M. P. 2016. *Process Mining - Data Science in Action, Second Edition*. Springer.
- van der Aalst, W. M. P., A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros. 2003. "Workflow Patterns". *Distributed Parallel Databases* 14(1):5–51.
- vom Brocke, J., W. M. P. van der Aalst, N. Berente, B. F. van Dongen, T. Grisold, W. Kremser, et al. 2024. "Process science: the interdisciplinary study of socio-technical change". *Process Sci.* 1(1).
- Wilsdorf, P., A. Wolpers, J. Hilton, F. Haack, and A. Uhrmacher. 2023. "Automatic Reuse, Adaption, and Execution of Simulation Experiments via Provenance Patterns". *ACM Transactions on Modeling and Computer Simulation* 33(1-2).
- Wynn, M. 2006. *Semantics, verification, and implementation of workflows with cancellation regions and OR-joins*. Ph. D. thesis.
- Zeigler, B. P., A. Muzy, and E. Kofman. 2018. *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations*. Elsevier.

AUTHOR BIOGRAPHIES

ADAM BANHAM is a Postdoctoral Research Fellow in the School of Computer Science and Information Technology at Adelaide University. His research interests include process science, process mining, business process management, conceptual modeling, and, more recently, the engineering and simulation of complex systems. His e-mail address is adam.banham@adelaide.edu.au and his website is <https://adambanham.io>.

CLAUDIA SZABO is a Professor in the School of Computer Science and Information Technology at Adelaide University. Her research interests lie in complex systems and in using simulation to identify and validate their emergent properties. Her e-mail address is claudia.szabo@adelaide.edu.au and her website is <https://researchers.adelaide.edu.au/profile/claudia.szabo>.